B.TECH COMPUTER SCIENCE ENGINEERING | PROJECT DEFENSE

PROJECT CATEGORY: FULL-STACK WEB APPLICATION

PICTURE PANNEL – SECURE DIGITAL MEDIA VAULT

Your Personal Media. Your Private Vault.

A password-protected vault to store, view and manage personal photos, videos and other supported content, using AES-GCM encrypted storage.

OBJECTIVE Securely store and manage personal photos and videos through password-protected vaults.

STUDENT NAME *[to be filled]*	ROLL NUMBER *[to be filled]*
DEPARTMENT *[to be filled]*	COLLEGE NAME *[to be filled]*
PROJECT GUIDE *[to be filled]*	ACADEMIC YEAR *[to be filled]*



Problem Statement and Motivation

Access protection
Personal photos and videos need better access protection.

Separate private spaces
Users may want separate private storage spaces.

Independent vaults
Ordinary media galleries do not necessarily provide independent password-protected vaults.

Simple interface
Users need an easy way to upload, access and manage their private media.

Ordinary media storage
General case

- Media may sit in one shared library
- Separate password vaults not guaranteed
- Per-file encryption not guaranteed
- Access control depends on the app or device

Picture Pannel vault
Proposed system

- Separate vault per password
- Password required to open a vault
- AES-GCM encrypted media storage
- Simple upload, view and manage UI

WHY PICTURE PANNEL?

Built to give users a simple, private, password-protected place for their photos and videos.

The comparison is conceptual and does not claim that other gallery applications are insecure.

Proposed Solution and Key Features

Password Vault Access
Create or open a vault with a password

Separate Vaults
Each password opens its own vault

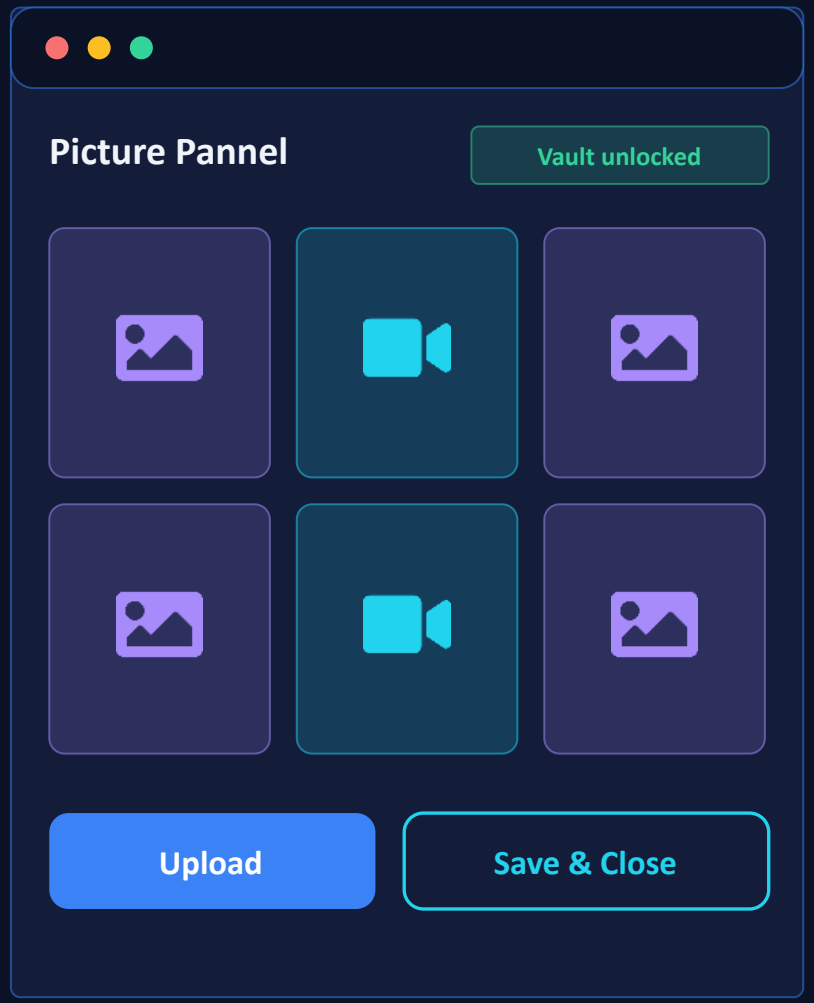
Photo & Video Upload
Add images and videos to an open vault

AES-GCM Encryption
Media is encrypted before it is stored

View & Manage Media
Browse and manage saved media

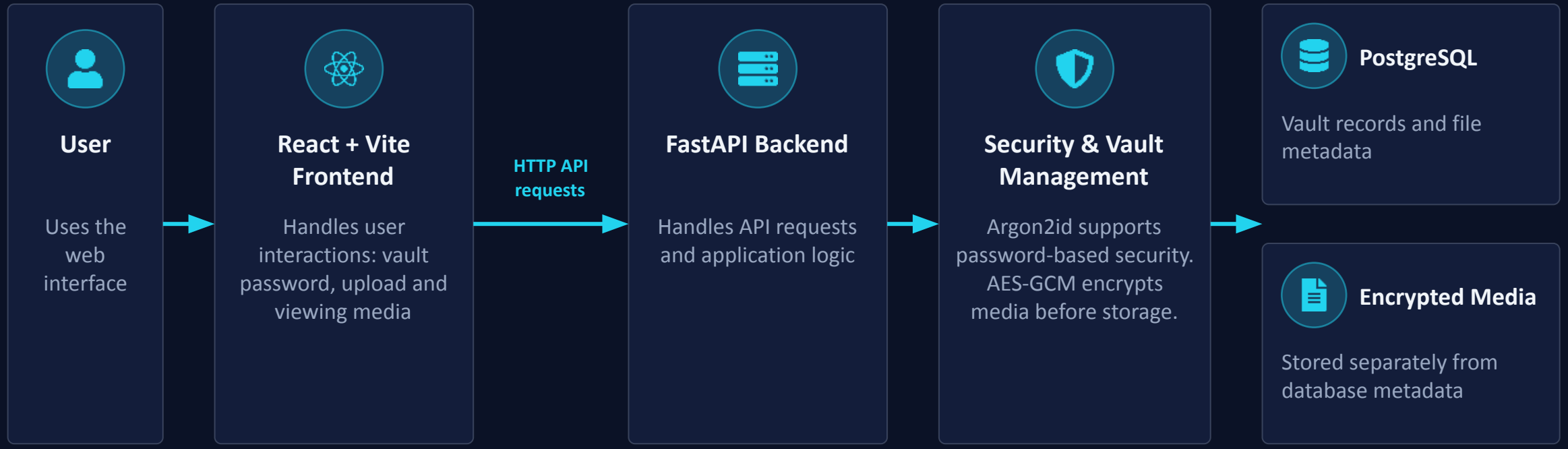
Save & Close Vault
Save content and close the vault

Backend API Communication
The React frontend communicates with the FastAPI backend over HTTP APIs.



Illustrative interface mockup

System Architecture



CURRENT IMPLEMENTATION

Encrypted media files are saved to local file storage on the backend. PostgreSQL stores vault records and file metadata.

PLANNED | NOT YET IMPLEMENTED

Supabase Storage or other cloud object storage for persistent encrypted media.

Working Process and System Workflow

UPLOAD AND STORE FLOW

1 Open the app
User opens Picture Pannel.

2 Enter password
User enters a vault password.

3 Open or create vault
Backend opens an existing vault or creates a new one.

4 Select media
User selects photos or videos.

5 Send to API
Frontend sends the media to FastAPI.

6 Validate & encrypt
Backend validates the media and encrypts it with AES-GCM.

7 Store & record
Encrypted media is stored; metadata saved in PostgreSQL.

8 View & close
Views media via authorized backend access; closes vault.

REVERSE ACCESS: RETRIEVE AND VIEW SAVED MEDIA

A Request media
Frontend asks the backend for a saved file.

B Authorize & fetch
Backend validates access and retrieves the encrypted file.

C Decrypt
Backend decrypts the media with AES-GCM after authorization.

D Return content
Backend returns the content to the frontend for display.

Security and Database Design

SECURITY MECHANISMS

Argon2id
Password-based key derivation and security

AES-GCM
Authenticated encryption of stored media

Unique Nonce
A random nonce for each encrypted file

Access Validation
Backend-side checks before media is served

Separate Storage
Encrypted media kept apart from DB metadata

Private Vaults
Vault opened only with its password

DATABASE DESIGN (SIMPLIFIED)

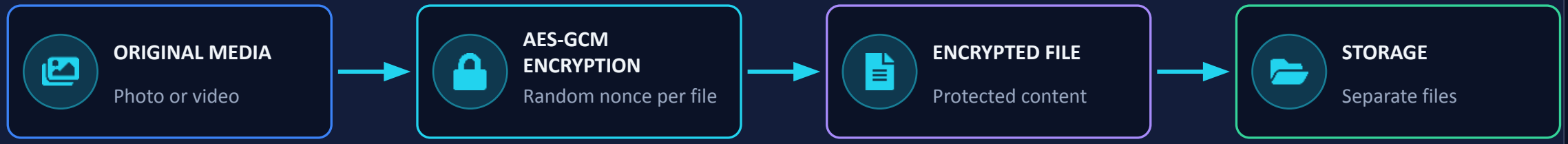
Vaults
One record per password-protected vault

Vault Sessions
Open-vault session records

Stored Files
File metadata and a reference to the encrypted file

ENCRYPTION WORKFLOW

PostgreSQL stores records and metadata. Encrypted media is stored in separate files.



Implementation, Testing and Results

IMPLEMENTATION

- React + Vite frontend (HTML, CSS, JS)
- FastAPI backend (Python)
- PostgreSQL integration via SQLAlchemy
- API endpoints tested using Swagger UI
- Encrypted photo and video upload workflow
- Git and GitHub for version control. Deployment target: Vercel (frontend) and Render (backend); live status to be validated.

#	Test case	Expected result	Status
1	Open or create vault	Vault access granted	To be validated
2	Upload supported image	Encrypted file saved	To be validated
3	Upload supported video	Encrypted file saved	To be validated
4	Retrieve saved media	Authorized media displayed	To be validated
5	Close and reopen vault	Access controlled by password	To be validated

KEY APPLICATION SCREENS (SCREENSHOT PLACEHOLDERS)

Vault password screen

[Insert screenshot]

Main dashboard

[Insert screenshot]

Media upload screen

[Insert screenshot]

Saved media gallery

[Insert screenshot]

Conclusion and Future Enhancements

CONCLUSION

Picture Pannel provides a simple password-protected digital media vault using modern full-stack technologies and encryption mechanisms.

- Password-based vault access
- AES-GCM encrypted media
- React, FastAPI and PostgreSQL

FUTURE ENHANCEMENTS

Cloud Storage

Supabase Storage or cloud object storage for persistent encrypted media

Better Authentication

Improved user authentication and account management

Mobile Support

Mobile-responsive UI and mobile application support

Search & Categories

Media search and categorization

Secure Sharing

Sharing with controlled access

Backup & Recovery

Backup and recovery features

Thank You